

Graph Basics

Sean

August 25, 2026



Outline

- Graph model and vocabulary
- Graph representations
- DFS and connected components
- BFS and shortest paths
- Grid and implicit graphs
- Multi-source BFS: Monsters



Why graphs?

- Graphs model objects and relationships

Why graphs?

- Graphs model objects and relationships
- Cities connected by roads, computers connected by cables, grid cells connected by moves

Why graphs?

- Graphs model objects and relationships
- Cities connected by roads, computers connected by cables, grid cells connected by moves
- Many problems become easier once we ask: what are the states, and what moves are allowed?

Why graphs?

- Graphs model objects and relationships
- Cities connected by roads, computers connected by cables, grid cells connected by moves
- Many problems become easier once we ask: what are the states, and what moves are allowed?
- This lecture focuses on storing graphs and traversing them without repeated visits

1 Graph model and vocabulary

2 Graph representations

3 DFS and connected components

4 BFS and shortest paths

5 Grid and implicit graphs

6 Multi-source BFS: Monsters



Graph model and vocabulary

What is a graph?

- A graph is a set of vertices and edges

What is a graph?

- A graph is a set of vertices and edges
- Vertices are the objects: $1, 2, 3, \dots, n$

What is a graph?

- A graph is a set of vertices and edges
- Vertices are the objects: $1, 2, 3, \dots, n$
- Edges are the connections between vertices

What is a graph?

- A graph is a set of vertices and edges
- Vertices are the objects: $1, 2, 3, \dots, n$
- Edges are the connections between vertices
- We usually write a graph as $G = (V, E)$

What is a graph?

- A graph is a set of vertices and edges
- Vertices are the objects: $1, 2, 3, \dots, n$
- Edges are the connections between vertices
- We usually write a graph as $G = (V, E)$
- In competitive programming, $n = |V|$ and $m = |E|$



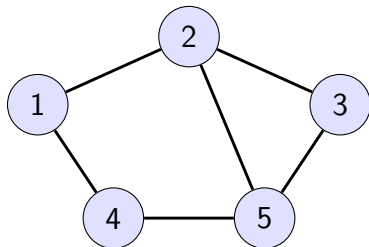
Modeling a problem as a graph

- Choose the vertices: what are the objects or states?
- Choose the edges: what direct moves or relationships are allowed?
- Decide direction and cost: can moves reverse, and do edges have weights?

Problem clue	Vertex	Edge
Cities and roads	City	Road between cities
Prerequisites	Course	Dependency arrow
Maze grid	Open cell	Neighbor step

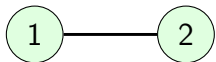


Example graph



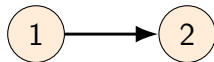
- $V = \{1, 2, 3, 4, 5\}$
- $E = \{(1, 2), (2, 3), (1, 4), (4, 5), (5, 3), (2, 5)\}$

Directed and undirected



Undirected

- Edge (1, 2) works both ways
- Roads, friendships, grid moves



Directed

- Edge (1, 2) means $1 \rightarrow 2$
- One-way roads, dependencies

Weighted and unweighted

- An unweighted edge only says that a move is allowed
- A weighted edge also has a cost
- Costs can mean distance, time, price, risk, or anything the problem defines
- Traversal basics are usually taught first on unweighted graphs
- Weighted shortest paths need stronger algorithms such as Dijkstra's algorithm



Common graph words

- Neighbor: a vertex directly connected by one edge
- Degree: in an undirected graph, the number of edges touching a vertex
- Path: a sequence of vertices connected by edges
- Cycle: a path that returns to its starting vertex
- Connected component: in an undirected graph, a maximal set of vertices where every pair can reach each other



- 1 Graph model and vocabulary
- 2 Graph representations
- 3 DFS and connected components
- 4 BFS and shortest paths
- 5 Grid and implicit graphs
- 6 Multi-source BFS: Monsters



Graph representations

Edge list

- Store every edge as a pair (u, v)

Edge list

- Store every edge as a pair (u, v)
- For weighted graphs, store triples (u, v, w)



Edge list

- Store every edge as a pair (u, v)
- For weighted graphs, store triples (u, v, w)
- Very simple and memory efficient: $\mathcal{O}(m)$



Edge list

- Store every edge as a pair (u, v)
- For weighted graphs, store triples (u, v, w)
- Very simple and memory efficient: $\mathcal{O}(m)$
- Good when the algorithm needs to process all edges



Edge list

- Store every edge as a pair (u, v)
- For weighted graphs, store triples (u, v, w)
- Very simple and memory efficient: $\mathcal{O}(m)$
- Good when the algorithm needs to process all edges
- Slow when we often ask: what are the neighbors of u ?



Edge list code

```
1  int n, m;
2  cin >> n >> m;
3
4  vector<pair<int, int>> edges;
5  for (int i = 0; i < m; i++) {
6      int u, v;
7      cin >> u >> v;
8      edges.push_back({u, v});
9  }
```



Adjacency matrix

- Store whether each pair (u, v) has an edge



Adjacency matrix

- Store whether each pair (u, v) has an edge
- Usually implemented as a 2D table: `adj[u][v]`



Adjacency matrix

- Store whether each pair (u, v) has an edge
- Usually implemented as a 2D table: `adj[u][v]`
- Edge existence query is fast: $\mathcal{O}(1)$



Adjacency matrix

- Store whether each pair (u, v) has an edge
- Usually implemented as a 2D table: `adj[u][v]`
- Edge existence query is fast: $\mathcal{O}(1)$
- Memory is expensive: $\mathcal{O}(n^2)$



Adjacency matrix

- Store whether each pair (u, v) has an edge
- Usually implemented as a 2D table: `adj[u][v]`
- Edge existence query is fast: $\mathcal{O}(1)$
- Memory is expensive: $\mathcal{O}(n^2)$
- Usually only practical when n is small or the graph is dense



Adjacency matrix code

```
1  int n, m;
2  cin >> n >> m;
3
4  vector<vector<bool>> adj(n + 1, vector<bool>(n + 1));
5  for (int i = 0; i < m; i++) {
6      int u, v;
7      cin >> u >> v;
8      adj[u][v] = true;
9      adj[v][u] = true; // remove this line for a directed graph
10 }
```



Adjacency list

- For each vertex, store the neighbors reachable by one edge



Adjacency list

- For each vertex, store the neighbors reachable by one edge
- This is the default representation for most graph traversal problems



Adjacency list

- For each vertex, store the neighbors reachable by one edge
- This is the default representation for most graph traversal problems
- Memory is $\mathcal{O}(n + m)$



Adjacency list

- For each vertex, store the neighbors reachable by one edge
- This is the default representation for most graph traversal problems
- Memory is $\mathcal{O}(n + m)$
- Iterating through all neighbors of u takes $\mathcal{O}(\deg(u))$



Adjacency list

- For each vertex, store the neighbors reachable by one edge
- This is the default representation for most graph traversal problems
- Memory is $\mathcal{O}(n + m)$
- Iterating through all neighbors of u takes $\mathcal{O}(\deg(u))$
- Iterating through every adjacency list takes $\mathcal{O}(n + m)$



Adjacency list code

```
1  int n, m;
2  cin >> n >> m;
3
4  vector<vector<int>> adj(n + 1);
5  for (int i = 0; i < m; i++) {
6      int u, v;
7      cin >> u >> v;
8      adj[u].push_back(v);
9      adj[v].push_back(u); // remove this line for a directed graph
10 }
```



Which representation?

- Edge list: process all edges directly
- Adjacency matrix: answer edge existence queries in $\mathcal{O}(1)$
- Adjacency list: traverse sparse graphs in $\mathcal{O}(n + m)$
- Default to adjacency list unless the problem gives a reason not to

- 1 Graph model and vocabulary
- 2 Graph representations
- 3 DFS and connected components**
- 4 BFS and shortest paths
- 5 Grid and implicit graphs
- 6 Multi-source BFS: Monsters



DFS and connected components

Traversal idea

- Traversal means visiting vertices by following edges

Traversal idea

- Traversal means visiting vertices by following edges
- The main danger is revisiting vertices forever

Traversal idea

- Traversal means visiting vertices by following edges
- The main danger is revisiting vertices forever
- Use a `visited` array to remember what has already been reached

Traversal idea

- Traversal means visiting vertices by following edges
- The main danger is revisiting vertices forever
- Use a `visited` array to remember what has already been reached
- In an undirected graph, one start vertex visits exactly its connected component

Traversal idea

- Traversal means visiting vertices by following edges
- The main danger is revisiting vertices forever
- Use a `visited` array to remember what has already been reached
- In an undirected graph, one start vertex visits exactly its connected component
- To cover the whole graph, try starting from every unvisited vertex



Depth-first search

- DFS explores one path as far as possible before backing up

Depth-first search

- DFS explores one path as far as possible before backing up
- Recursion naturally matches this explore-and-return pattern



Depth-first search

- DFS explores one path as far as possible before backing up
- Recursion naturally matches this explore-and-return pattern
- It is useful for components, cycle detection, topological sorting, and many tree problems



Depth-first search

- DFS explores one path as far as possible before backing up
- Recursion naturally matches this explore-and-return pattern
- It is useful for components, cycle detection, topological sorting, and many tree problems
- On an adjacency list, DFS runs in $\mathcal{O}(n + m)$

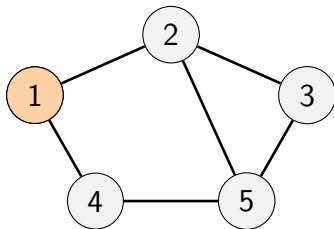


DFS code

```
1  vector<vector<int>> adj;
2  vector<bool> visited;
3
4  void dfs(int u) {
5      visited[u] = true;
6      for (int v : adj[u]) {
7          if (!visited[v]) {
8              dfs(v);
9          }
10     }
11 }
```

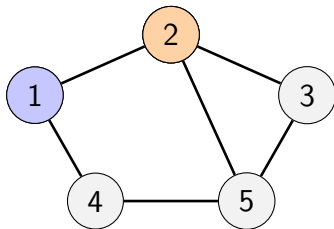


DFS trace



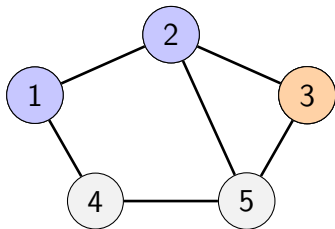
- Start at vertex 1

DFS trace



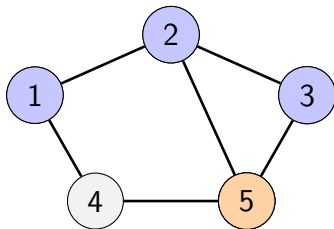
- Start at vertex 1
- Go deeper to 2

DFS trace



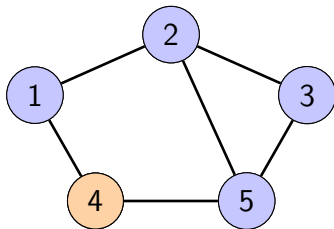
- Start at vertex 1
- Go deeper to 2
- Go deeper to 3

DFS trace



- Start at vertex 1
- Go deeper to 2
- Go deeper to 3
- From 3, continue to 5

DFS trace



- Start at vertex 1
- Go deeper to 2
- Go deeper to 3
- From 3, continue to 5
- From 5, finish with 4

DFS order: 1, 2, 3, 5, 4

Counting components

```
1  int components = 0;
2  visited.assign(n + 1, false);
3
4  for (int u = 1; u <= n; u++) {
5      if (!visited[u]) {
6          components++;
7          dfs(u);
8      }
9  }
```

DFS notes

- Recursive DFS is concise and easy to write
- Very long paths can make recursion depth a problem
- In that case, use iterative DFS with `stack<int>`
- The same `visited` idea still prevents repeated visits



- 1 Graph model and vocabulary
- 2 Graph representations
- 3 DFS and connected components
- 4 BFS and shortest paths**
- 5 Grid and implicit graphs
- 6 Multi-source BFS: Monsters



BFS and shortest paths

Breadth-first search

- BFS explores vertices by increasing edge count from the start



Breadth-first search

- BFS explores vertices by increasing edge count from the start
- It uses a queue



Breadth-first search

- BFS explores vertices by increasing edge count from the start
- It uses a queue
- First visit the start vertex, then its neighbors, then their neighbors, and so on



Breadth-first search

- BFS explores vertices by increasing edge count from the start
- It uses a queue
- First visit the start vertex, then its neighbors, then their neighbors, and so on
- On an adjacency list, BFS runs in $\mathcal{O}(n + m)$

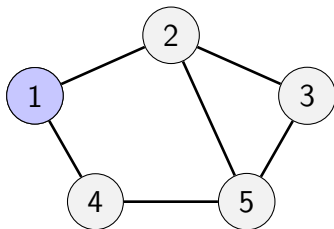


BFS code

```
1  vector<int> dist(n + 1, -1);
2  queue<int> q;
3
4  dist[start] = 0;
5  q.push(start);
6
7  while (!q.empty()) {
8      int u = q.front();
9      q.pop();
10
11     for (int v : adj[u]) {
12         if (dist[v] == -1) {
13             dist[v] = dist[u] + 1;
14             q.push(v);
15         }
16     }
17 }
```

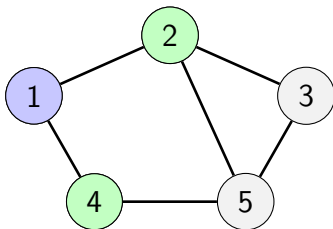


BFS trace



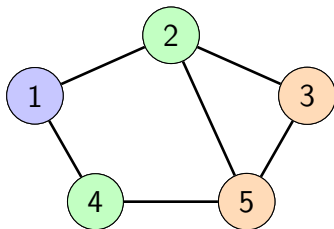
- Layer 0: start at 1

BFS trace



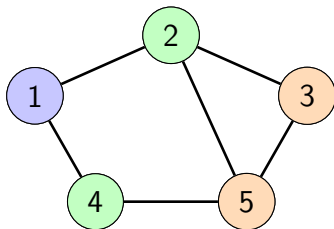
- Layer 0: start at 1
- Layer 1: discover 2 and 4

BFS trace



- Layer 0: start at 1
- Layer 1: discover 2 and 4
- Layer 2: discover 3 and 5

BFS trace



- Layer 0: start at 1
- Layer 1: discover 2 and 4
- Layer 2: discover 3 and 5
- First visit gives the smallest edge count

Why BFS gives shortest paths

- The queue processes vertices in layers

Why BFS gives shortest paths

- The queue processes vertices in layers
- Layer 0: the starting vertex



Why BFS gives shortest paths

- The queue processes vertices in layers
- Layer 0: the starting vertex
- Layer 1: vertices one edge away



Why BFS gives shortest paths

- The queue processes vertices in layers
- Layer 0: the starting vertex
- Layer 1: vertices one edge away
- Layer 2: vertices two edges away



Why BFS gives shortest paths

- The queue processes vertices in layers
- Layer 0: the starting vertex
- Layer 1: vertices one edge away
- Layer 2: vertices two edges away
- The first time BFS reaches a vertex, it has found a path with the fewest edges



Recovering a path

```
1  vector<int> parent(n + 1, -1);
2
3  // When BFS moves from u to v:
4  parent[v] = u;
5
6  vector<int> path;
7  for (int cur = target; cur != -1; cur = parent[cur]) {
8      path.push_back(cur);
9  }
10 reverse(path.begin(), path.end());
```



When to use DFS or BFS

- DFS: reachability, components, cycles, and structural checks
- BFS: fewest-edge distances in an unweighted graph
- Both need a graph representation and a way to mark visited vertices
- Both are linear in the graph size when using adjacency lists



DFS or BFS decision table

Task	Pick	Why
Reachability	Either	Reachable vertices
Components	Either	Repeat unvisited
Fewest edges	BFS	Layer order
Structure checks	DFS	Branch-first order
Deep recursion risk	Iterative DFS	Explicit stack



- 1 Graph model and vocabulary
- 2 Graph representations
- 3 DFS and connected components
- 4 BFS and shortest paths
- 5 Grid and implicit graphs**
- 6 Multi-source BFS: Monsters



Grid and implicit graphs

Grid graph idea

(0, 0)	(0, 1)	(0, 2)
(1, 0)	(1, 1)	(1, 2)
(2, 0)	(2, 1)	(2, 2)

- A cell (r, c) is a vertex
- Edges are allowed moves between cells
- Four-direction moves use up, down, left, and right
- Check bounds and blocked cells before moving

Implicit graphs

- We do not need to store every grid edge ahead of time
- A cell's neighbors are generated when DFS or BFS visits it
- The arrays `dr` and `dc` are the edge generator

```
1  int dr[4] = {-1, 1, 0, 0};
2  int dc[4] = {0, 0, -1, 1};
3
4  for (int k = 0; k < 4; k++) {
5      int nr = r + dr[k];
6      int nc = c + dc[k];
7      // edge: (r, c) -> (nr, nc), if the cell is valid
8  }
```



Grid BFS neighbor loop

```
1  int dr[4] = {-1, 1, 0, 0};
2  int dc[4] = {0, 0, -1, 1};
3
4  for (int k = 0; k < 4; k++) {
5      int nr = r + dr[k];
6      int nc = c + dc[k];
7
8      if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
9      if (grid[nr][nc] == '#') continue;
10     if (visited[nr][nc]) continue;
11
12     visited[nr][nc] = true;
13     q.push({nr, nc});
14 }
```



Recognizing grid problems

- Counting regions: run DFS or BFS from every unvisited open cell
- Shortest path in a maze: use BFS from the start cell
- Several starting points at once: put all starts in the BFS queue
- Obstacles are cells that should not become vertices
- If moves all cost 1, BFS gives the shortest path length



- 1 Graph model and vocabulary
- 2 Graph representations
- 3 DFS and connected components
- 4 BFS and shortest paths
- 5 Grid and implicit graphs
- 6 Multi-source BFS: Monsters**



Multi-source BFS: Monsters

Example: Monsters

CSES: Monsters

You are in a labyrinth with walls, one player, and many monsters. Every move takes one step. Escape to a boundary square without ever sharing a square with a monster.

Example: Monsters

CSES: Monsters

You are in a labyrinth with walls, one player, and many monsters. Every move takes one step. Escape to a boundary square without ever sharing a square with a monster.

- Treat every monster as a BFS source

Example: Monsters

CSES: Monsters

You are in a labyrinth with walls, one player, and many monsters. Every move takes one step. Escape to a boundary square without ever sharing a square with a monster.

- Treat every monster as a BFS source
- Compute the earliest monster time for each cell

Example: Monsters

CSES: Monsters

You are in a labyrinth with walls, one player, and many monsters. Every move takes one step. Escape to a boundary square without ever sharing a square with a monster.

- Treat every monster as a BFS source
- Compute the earliest monster time for each cell
- This is still an implicit grid graph; the safety rule changes



Example: Monsters

CSES: Monsters

You are in a labyrinth with walls, one player, and many monsters. Every move takes one step. Escape to a boundary square without ever sharing a square with a monster.

- Treat every monster as a BFS source
- Compute the earliest monster time for each cell
- This is still an implicit grid graph; the safety rule changes
- Then run player BFS, only entering cells where the player arrives first



Example: Monsters

CSES: Monsters

You are in a labyrinth with walls, one player, and many monsters. Every move takes one step. Escape to a boundary square without ever sharing a square with a monster.

- Treat every monster as a BFS source
- Compute the earliest monster time for each cell
- This is still an implicit grid graph; the safety rule changes
- Then run player BFS, only entering cells where the player arrives first
- Store parent moves so a valid escape path can be printed



Monsters sample

Input

```
5 8
#####
#M..A..#
#.#.M#.#
#M#..#..
#.#####
```

One valid output

```
YES
5
RRDDR
```

- The player starts at A; monsters start at every M
- The path must stay safe even if monsters know it in advance



Monster times first

```
1  const int INF = 1e9;
2  vector<vector<int>> mt(rows, vector<int>(cols, INF));
3  queue<pair<int,int>> q;
4  for (auto [r, c] : monsters) {
5      mt[r][c] = 0;
6      q.push({r, c});
7  }
8  while (!q.empty()) {
9      auto [r, c] = q.front();
10     q.pop();
11     for (auto [nr, nc] : neighbors(r, c)) {
12         if (wall(nr, nc) || mt[nr][nc] != INF) continue;
13         mt[nr][nc] = mt[r][c] + 1;
14         q.push({nr, nc});
15     }
16 }
```



Player BFS with safety

```
1  vector<vector<int>> d(rows, vector<int>(cols, -1));
2  vector<vector<char>> par(rows, vector<char>(cols));
3  queue<pair<int,int>> q;
4  pair<int,int> exit = {-1, -1};
5  d[sr][sc] = 0;
6  q.push({sr, sc});
7  while (!q.empty()) {
8      auto [r, c] = q.front();
9      q.pop();
10     if (on_border(r, c)) { exit = {r, c}; break; }
11     for (auto [nr, nc, ch] : next_cells(r, c)) {
12         int nd = d[r][c] + 1;
13         if (wall(nr, nc) || d[nr][nc] != -1) continue;
14         if (nd >= mt[nr][nc]) continue;
15         d[nr][nc] = nd;
16         par[nr][nc] = ch;
17         q.push({nr, nc});
18     }
19 }
```



Recovering a grid path

```
1  string path;
2  auto [r, c] = exit;
3
4  while (r != sr || c != sc) {
5      char ch = par[r][c];
6      path.push_back(ch);
7
8      if (ch == 'U') r++;
9      else if (ch == 'D') r--;
10     else if (ch == 'L') c++;
11     else if (ch == 'R') c--;
12 }
13
14 reverse(path.begin(), path.end());
```



Monsters implementation checklist

- Run monster BFS before player BFS
- Initialize every monster as distance 0 in the same queue
- Use strict safety: player time must be less than monster time
- If the player starts on the border, the path length is 0
- Check escape when the player reaches any border cell
- Store parent moves so the escape path can be printed



Common graph mistakes

- Forgetting visited: DFS or BFS can revisit vertices forever
- Marking visited too late in BFS: one vertex can enter the queue many times
- Mixing 0-indexed code with 1-indexed input
- Treating directed edges as if they work both ways
- Using BFS for weighted shortest paths



Practice problems

- CSES: Counting Rooms – connected components on a grid
- CSES: Building Roads – connected components in an undirected graph
- CSES: Message Route – BFS shortest path in an unweighted graph
- CSES: Labyrinth – BFS shortest path on a grid
- CSES: Monsters – multi-source BFS plus path recovery



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

